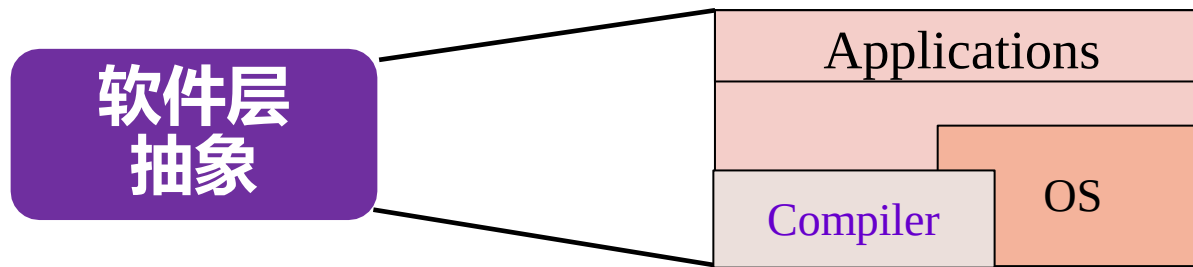
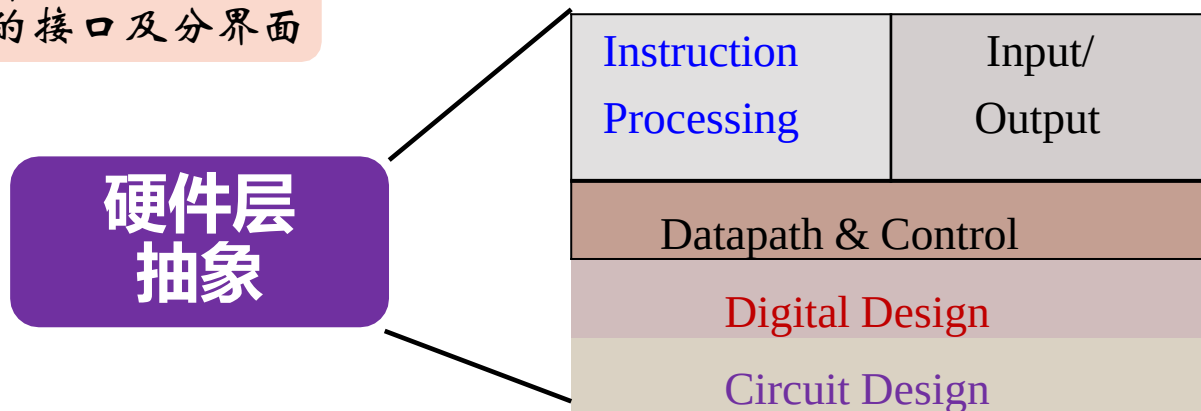


3. 指令系统与汇编语言



指令系统

计算机硬件和软件的接口及分界面



指令：计算机能够理解的语言

指令系统/指令集：所有指令构成的集合，也可以直接称为机器语言

指令系统与汇编语言

- 指令系统/机器语言：二进制形式
- 汇编语言：英文助记符表示，便于记忆和编写。
 - 检查和改写汇编代码，提高程序效率；
 - 增强对硬件的直接控制能力

主要内容

- 3.1 指令系统的发展
- 3.2 指令格式
- 3.3 寻址方式
- 3.4 指令功能/类型
- 3.5 MIPS汇编程序

3.1 指令系统的发展

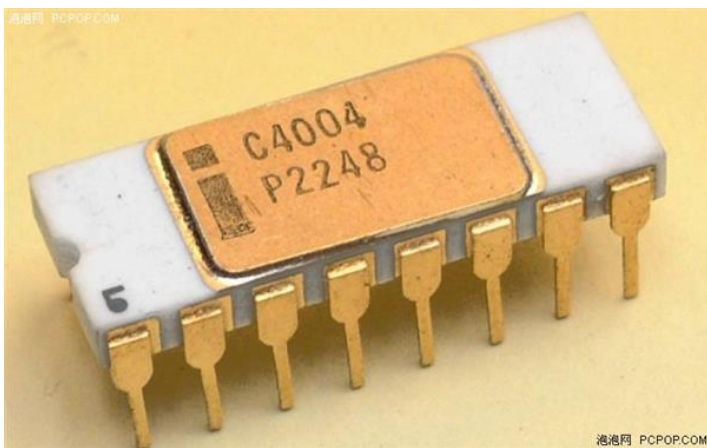
1. CPU和指令集
2. CISC和RISC指令系统
3. IA-32、X86-64的发展
4. ARM指令系统的发展

3.1.1 CPU和指令集

- 中央处理单元 CPU
 - 以前的分离元件集中到集成电路上实现
 - 主要包括运算器、控制器、寄存器（可看做存储器的一部分）、高速缓存
 - 实现指令集（算术逻辑运算、存储器访问、分支跳转等指令）
- 不同的CPU支持不同的指令集
 - 指令集架构：Instruction Set Architecture (ISA).
 - 主要的ISA：Intel 80x86, IBM/Motorola PowerPC, MIPS, ARM, RISC-V...
 - *MIPS*: Microprocessor without Interlocked Pipelines Stages, 创立于80年代

X86 CPU的发展历史

- 1957年，从“晶体管之父”肖克利（获得诺贝尔物理学奖）实验室出走的“八叛徒”创立仙童公司（Fairchild），发明集成电路
- Intel成立于1968年，创始人包括来自仙童的罗伯特·诺伊斯，葛罗夫（A. Grove）和戈登·摩尔（Gordon Moore）；AMD成立于1969年，创始人是仙童的销售杰里·桑德斯（Jerry Sanders）
- 1971年第一款4位微处理器Intel 4004问世，1974年第一款8位微处理器Intel 8080问世



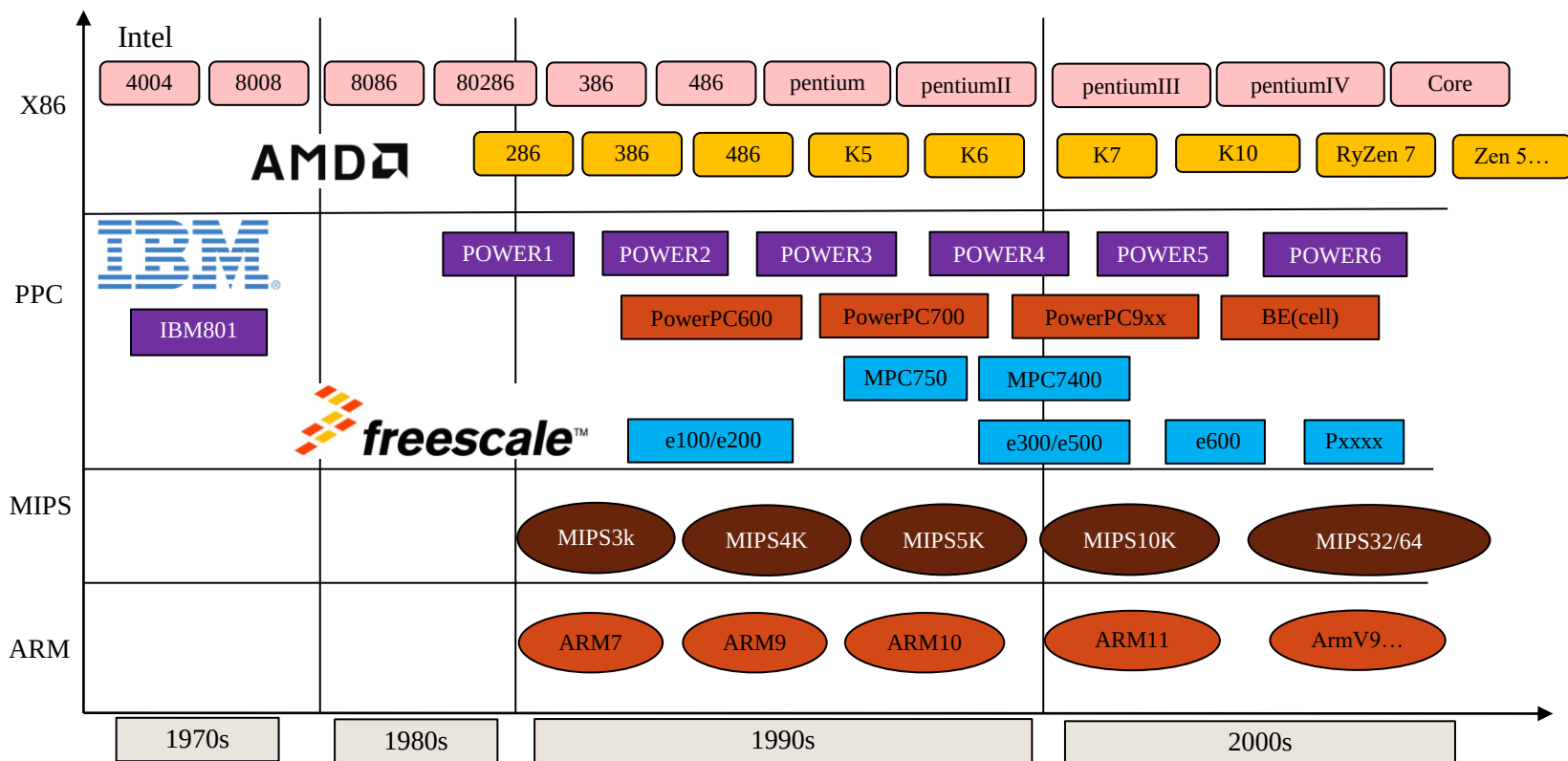
Intel 4004是第一个商用微处理器



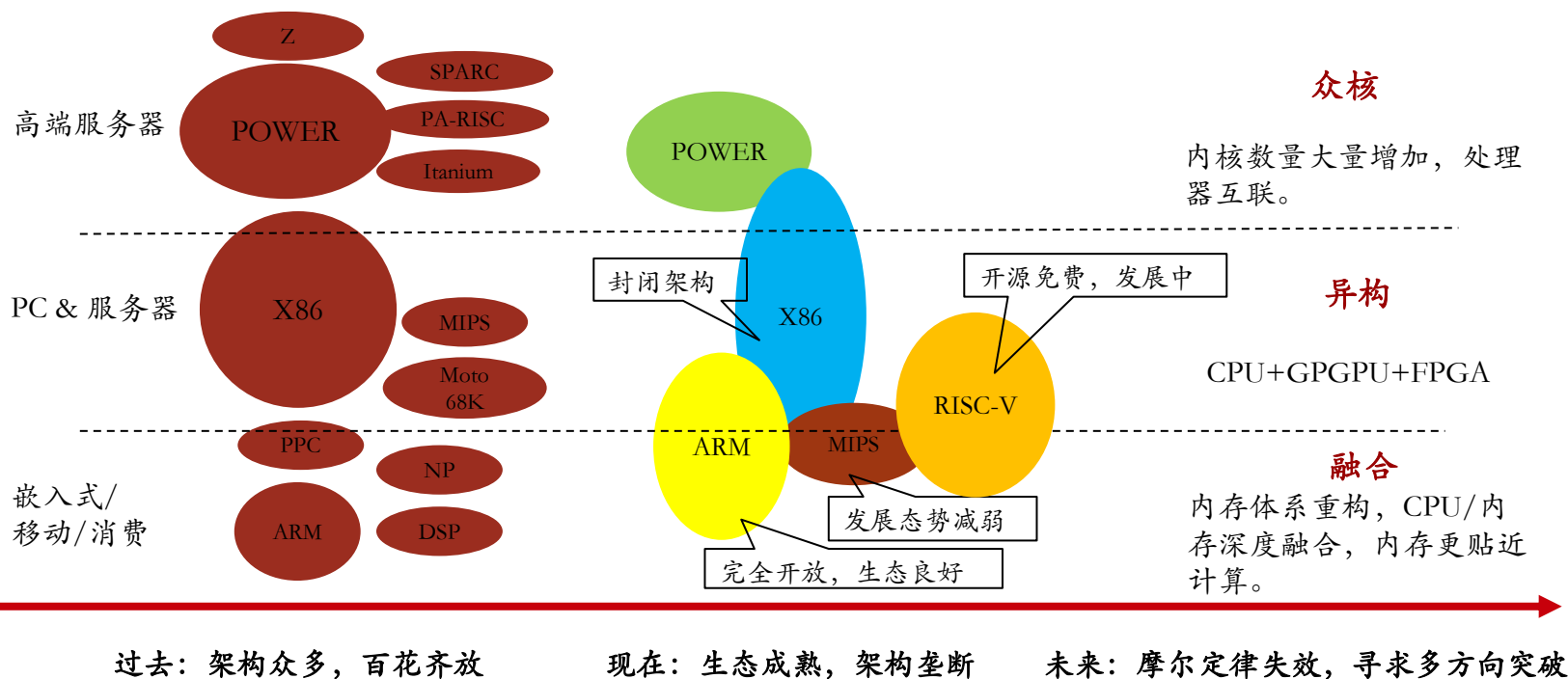
采用4004的Busicom Unicom计算器

- 1978年, Intel推出了8086处理器, 成为个人计算机的标准平台, 是最为成功的CPU架构
- 1982年, 在IBM公司的要求下, AMD获得Intel授权, 成为其8086、8088芯片的第二供应商
- 1986年, Intel中止了与AMD的合作, 二者展开了8年的法律大战
- 1993年 - Intel 推出了Pentium处理器, 广泛应用于个人电脑中。
- 1995年, AMD独立开发发布K5, 与Intel的Pentium PK; 后来发布K6、K7 (速龙), 在主频上超过Intel
- 2003年, AMD发布首款支持64位架构的处理器
- 2006年, Intel的Core 2发布, 性能明显增长, 全面领先AMD
- 2011年 - AMD将CPU和GPU核心整合到同一芯片上, 称为APU (Accelerated Processing Unit), 开启了异构计算的新时代
- 2017年, AMD锐龙 Ryzen 7处理器的亮相, 率先采用先进的7nm制程, 性能全面超越同期英特尔产品
- 2022年: AMD发布首款搭载 3D V-Cache 堆叠缓存技术的 锐龙7 5800X3D, 成为当时最强的游戏CPU
- 2024年: AMD在x86 CPU市场的份额首次超过英特尔,

主流CPU发展路径



处理器发展趋势



3.1.2 CISC和RISC指令系统

- 两种指令系统设计方向：CISC & RISC
- 增强指令功能：CISC (Complex Instruction Set Computer) , 即复杂指令系统计算机
 - 特点：格式复杂，寻址方式复杂，指令长度不定；把一些原来由软件实现的、常用的功能改用硬件的指令系统来实现。
 - 实例：80X86指令系统
- 简化指令功能：RISC (Reduced Instruction Set Computer) , 即精简指令系统计算机
 - 特点：格式简单，指令长度和操作码长度固定；简单寻址方式，大部分指令使用寄存器直接寻址。
 - 实例：Power, ARM, MIPS , RISC-V指令系统

CISC

- CISC发展的背景

- 计算机硬件成本的不断下降，软件开发成本的不断提高
 - 在指令系统中增加更多的、更复杂的指令，以提高操作系统的效率，并尽量缩短指令系统与高级语言的语义差别，以便于高级语言的编译。
- 程序的兼容性：同一系列计算机的新机器和高档机的指令系统只能扩充而不能缩减。

- CISC指令系统的特点

- 指令系统复杂庞大(一般数百条指令)；
- 寻址方式多，指令格式多，指令字长不固定；
- 可访存指令不受限制；
- 各种指令使用频率相差很大；
- 各种指令执行时间相差很大；
- 大多数采用微程序控制器

TOP 10 80X86 INSTRUCTIONS

Rank	instruction	Integer Average Percent total executed
1	load	22%
2	conditional branch	20%
3	compare	16%
4	store	12%
5	add	8%
6	and	6%
7	sub	5%
8	move register-regi	4%
9	call	1%
10	return	1%
	Total	95%

(简单指令占主要部分，使用频率高！)

RISC

- RISC的产生背景

- 对CISC的测试表明：最长使用的是一些简单指令，占指令总数的20%，但在程序中出现的频率却占80%以上。而占20%的复杂指令，为实现其功能而设计的微程序代码却占总代码的80%。
- 复杂的指令系统需要复杂的控制器，占用较多的芯片面积，CISC研制时间长、成本高、实现流水线更加复杂。

- RISC指令系统的特点

- 优先选取使用频率最高的一些简单指令；
- 指令长度固定；
- 只有取数/存数指令（load/store）访问内存；
- CPU中的寄存器数量很多；
- 硬布线控制逻辑为主，不用或少用微码控制；加上优化编译配合硬件的改进，使系统的速度大大提高。
- 高效的流水线结构、延迟转移、重叠寄存器窗口技术等。

RISC & CISC

对比项目 \ 类别	CISC	RISC
指令系统	复杂，庞大	简单，精简
指令数目	一般大于 200 条	一般小于 100 条
指令字长	不固定	定长
可访存指令	不加限制	只有 Load/Store 指令
各种指令执行时间	相差较大	绝大多数在一个周期内完成
各种指令使用频度	相差很大	都比较常用
通用寄存器数量	较少	多
目标代码	难以用优化编译生成高效的目标代码程序	采用优化的编译程序，生成代码较为高效
控制方式	绝大多数为微程序控制	绝大多数为组合逻辑控制
指令流水线	可以通过一定方式实现	必须实现

■ RISC与CISC技术的融合

- 随着芯片集成度和硬件速度的增大，RISC系统也越来越复杂。
- CISC也吸收了很多RISC的设计思想，大多数常见指令被动态翻译成 μ ops，然后进行乱序流水线执行；非常复杂指令，调用一段由微指令组成的微程序。

3.1.3 IA-32、X-64的发展

- Intel的IA-32来源于早期的x86架构，IA-32、IA-64的主要发展历史：
 - 1978：Intel 8086发布，它扩展自之前已经成功的8位微处理器Intel 8080，是16位计算机。8086不是一个通用寄存器的体系结构，它的寄存器都是专用的。
 - 1980：Intel 8007浮点数协处理器问世，扩展了8086，并多了60个浮点数指令。
 - 1982：80286对8086做了扩展，将地址空间扩大到24位，创建了保护模式。
 - 1985：80386扩展80286的地址空间到32位，增加了一些新的寻址模式。
 - 1985~1995：添加4条指令。1989年发布80486，1992年发布Pentium。
 - 1997：多媒体扩展MMX指令集（Multi Media Extension），包括57条指令。
 - 1999：Intel 推出IA-64指令集，与IA-32不兼容，缺乏生态支持
 - 1999：AMD改进了IA-32体系结构，发布了AMD64位体系结构。
 - 2004：Intel 推出支持AMD64的Xeon 处理器
 - 2003至今： x64 /AMD64成为PC、服务器的主流架构

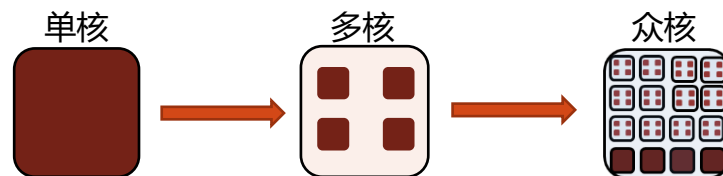
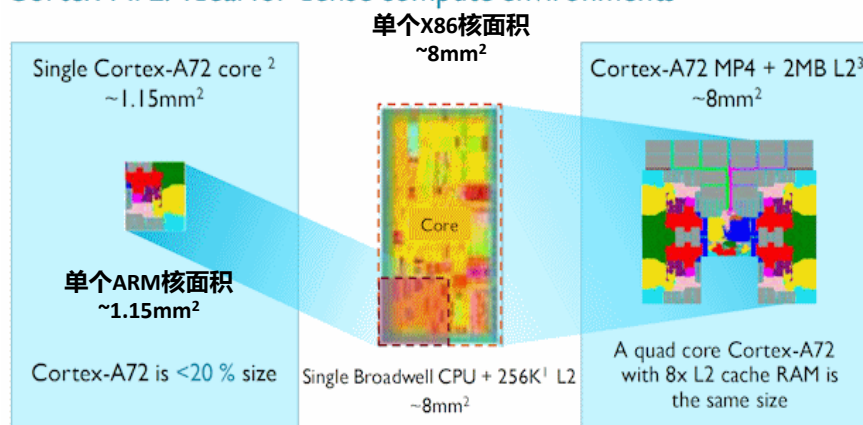
ARM处理器架构

- 基于RISC (精简指令集计算机), 定长指令集 (32位/64位) , 多个通用寄存器
- 主要架构版本
 - ARMv6: 引入了单指令多数据 (SIMD) 运算, 适用于高性能设备。
 - ARMv7: 支持Thumb-2技术, 分为Cortex-A (应用处理器)、Cortex-R (实时处理器) 和Cortex-M (微控制器) 。
 - ARMv8: 增加了64位支持, 同时兼容32位指令。
 - ARMv9: 增强了AI计算和安全性, 支持矢量计算和机器学习。

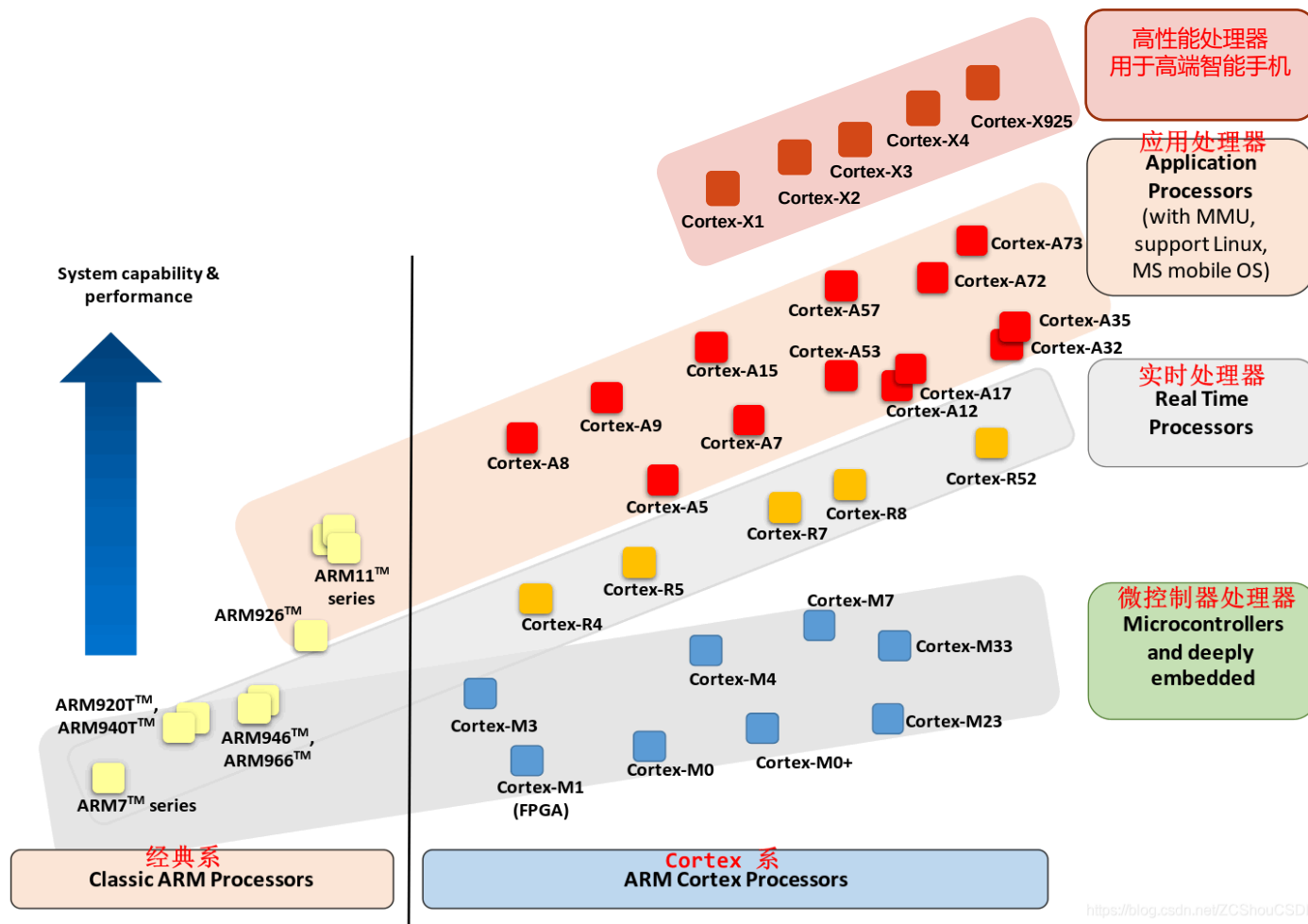
ARM提供更多计算核心

- 工艺、主频遇到瓶颈后，开始通过增加核数的方式来提升性能；
- 芯片的物理尺寸有限制，不能无限制的增加；
- ARM的众核横向扩展空间优势明显。

Cortex-A72: Ideal for dense compute environments



ARM处理器发展史



<https://blog.csdn.net/ZCShouCSDN>

主要内容

■ 3.1 汇编语言和指令系统的发展

■ **3.2 指令格式**

■ 3.3 寻址方式

■ 3.4 指令功能/类型

■ 3.5 MIPS汇编程序

3.2 指令格式

- 一条指令应包含（或隐含）的信息：
 1. 操作码：具体说明了操作的性质及功能
 2. 操作数的地址：可能有多个（主(虚)存，寄存器，I/O端口，指令本身)
 3. 操作结果的存储地址：主(虚)存，寄存器，I/O端口
 4. 下一条指令的地址：
 - 缺省按顺序依次执行的，无需显式指明
 - 少数指令不一定顺序执行，需要显式指明下一条指令的地址
- 指令格式：
 - **操作码**：表示该指令所要完成的操作
 - **地址码**：描述该指令的操作对象，或者直接给出操作数或者指出操作数的存储器地址或寄存器地址（即寄存器名）

操作码

地址码

指令格式的设计

- 指令格式的选择应遵循的几条基本原则
 - 应尽量短
 - 要有足够的操作码位数
 - 指令编码必须有唯一的解释，否则机器不知道该如何执行指令
 - 指令字长应是字节的整数倍
 - 合理地选择地址字段的个数
 - 指令尽量规整

指令格式的设计

- 与指令集设计相关的重要方面
 - 指令格式：指令长度，各字段长度，地址码个数
 - 操作码的全部组成：操作码个数，种类，复杂度
 - 数据类型：对哪几种数据类型完成操作
 - 通用寄存器：个数，功能，长度
 - 寻址方式：操作数地址的指定方式
 - 下条指令的地址如何确定：顺序，PC+1；条件转移；无条件转移；.....
 - 异常和中断机制，包括存储保护方式等

操作码

- 操作码：指明指令所要完成的操作，如加法、减法、传送、移位、转移等。
- 操作码的长度可以是固定的，也可以是变化的。
 - **固定长度**：格式单一，硬件设计比较简单，指令译码时间短。
 - 操作码所占的位数反映了机器最多能够支持的指令类型数量，例如操作码为7位，则该计算机最多可以包括 $2^7=128$ 条指令。
 - 广泛用于字长较长的高性能计算机，以及RISC（精简指令集计算机）指令系统中。
 - **变长**：操作码的编码长度分成几种固定长的格式，操作码的位数随地址数的减少而增加，例如X86架构。
 - 减少指令长度和程序总长度
 - 增加指令字所能表示的操作信息
 - 在指令字长有限的前提下，提供比较丰富的指令种类
 - 会增加指令译码和分析的难度，使控制器的设计复杂化

变长操作码设计的一个原则：
使用频度高的指令：短的操作码
使用频度低的指令：较长的操作码

地址码

■ 操作数的位置

- 立即数（内嵌在指令中）
- 寄存器（寄存器地址）
- 存储器（存储器地址）
- 输入输出端口（输入输出端口地址）

■ 操作数的类型

- 数值（无符号、定点、浮点）
- 逻辑型数、字符
- 地址（操作数地址、指令地址）

■ 操作数的存储方式

- 大端编址（big-endian）：最高有效字节存储在地址最小位置，字地址为最高位（左边）字节地址
- 小端编址（little-endian）：最低有效字节存储在地址最小位置，字地址为最低位（右边）字节地址

例：Int a; //0x12345678

地址	值
a+0	12
a+1	34
a+2	56
a+3	78

大端次序

地址	值
a+0	78
a+1	56
a+2	34
a+3	12

小端次序

地址码

- 地址个数
 - 地址个数越少，指令长度越短，指令功能越简单
 - 地址个数越少，所需指令条数越多，增加了程序复杂度和执行时间
- 地址的选择依赖于指令系统的结构
 - 堆栈结构：零地址指令
 - 累加器结构：一地址指令
 - 通用寄存器结构：二、三地址指令

地址码

■ 零地址指令

OP

- 无需操作数。如：空操作 / 停机等
- 所需操作数为默认的。如：堆栈等

■ 一地址指令

OP

A1

- 其地址既是源操作数地址，也是存放结果地址
- 单目运算：如：取反 / 取负，加一
- 双目运算：另一操作数为默认的，如：累加器

地址码

- **二地址指令(最常用)**

OP	A1	A2
----	----	----

- 分别存放双目运算中两个源操作数地址，并将其中一个地址作为存放结果地址

- **三地址指令(RISC风格)**

OP	A1	A2	A3
----	----	----	----

- 分别为双目运算中两个源操作数地址和一个结果地址

- **多地址指令**

- 用于成批数据处理的指令，如：向量指令 等

地址码

例： $Y = (A - B) \div (C + D \times E)$

二地址指令

指令		操 作
MOVE	Y,A	$Y \leftarrow (A)$
SUB	Y,B	$Y \leftarrow (Y) - (B)$
MOVE	T,D	$T \leftarrow (D)$
MUL	T,E	$T \leftarrow (T) \times (E)$
ADD	T,C	$T \leftarrow (T) + (C)$
DIV	Y,T	$Y \leftarrow (Y) \div (T)$

一地址指令

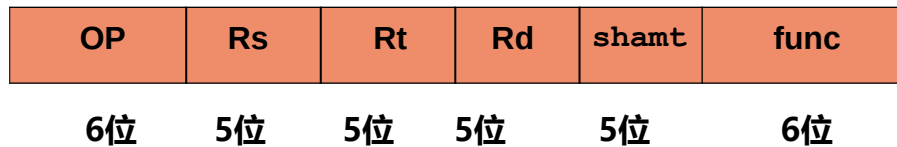
指令		操 作
LOAD	D	$AC \leftarrow (D)$
MUL	E	$AC \leftarrow (AC) \times (E)$
ADD	C	$AC \leftarrow (AC) + (C)$
STOR	Y	$Y \leftarrow (AC)$
LOAD	A	$AC \leftarrow (A)$
SUB	B	$AC \leftarrow (AC) - (B)$
DIV	Y	$AC \leftarrow (AC) \div (Y)$
STOR	Y	$Y \leftarrow (AC)$

MIPS指令集的格式

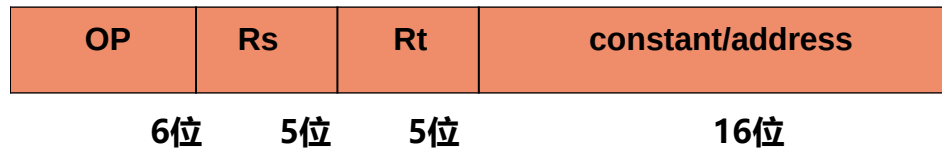
- 指令长度：固定为32位（4个字节）
 - 在MIPS中，一般将32位的数字称为“字”，16位也被称为半字。
- 按指令功能类型，分三类：R型、I型和J型。
 - 例如：add a, b, c # The sum of b and c is placed into a.

MIPS指令格式

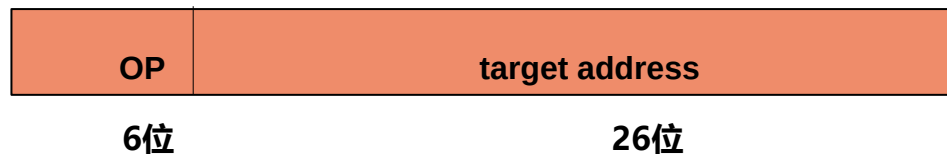
- R(register)类型指令



- I(immediate)类型指令

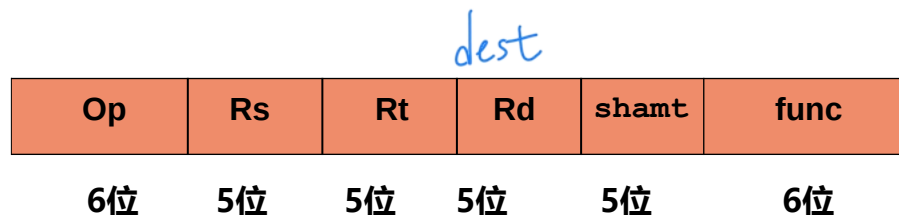


- J(jump)类型指令



R(REGISTER)类型指令

- 该类型指令从寄存器堆中读取两个源操作数，计算结果写回寄存器堆中的某个寄存器
- 这类指令是MIPS指令集中数量最多的一类



- op: 操作码，标识指令的功能；
- rs: 第一个源操作数，在R型指令中为寄存器；
- rt: 第二个源操作数，在R型指令中为寄存器；
- rd: 存放该条指令运算结果的目的地操作数，在R型指令中为寄存器；
- shamt: 位移量，一般指令中较少使用，典型应用是在移位指令中使用
- func: 功能码，实际上可以理解为op的扩展，用来标识指令功能。

MIPS指令中的寄存器

- 寄存器操作数字段的长度：5位
 - 支持的寄存器数量是32 ($=2^5$) 个。
- 表示一个寄存器：`\$`符后面跟两个字符
 - 用\$*s*0, \$*s*1, \$*s*2, ...表示程序中的变量对应的寄存器
 - 用\$*t*0, \$*t*1, \$*t*2, ...表示将程序编译为MIPS指令时所需的临时寄存器

名称	编号	功能
zero	0	常数0
at	1	为汇编程序保留
v0-v1	2-3	过程调用返回值
a0- a3	4-7	过程调用参数
t0-t7	8-15	临时变量
s0-s7	16-23	保存寄存器
t8- t9	24-25	其他临时变量
k0-k1	26-27	为OS保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

R(REGISTER)类型指令

- 典型的R型指令如加法指令add和减法指令sub
 - add和sub指令的op字段都为0，但是func字段的值分别为32和34

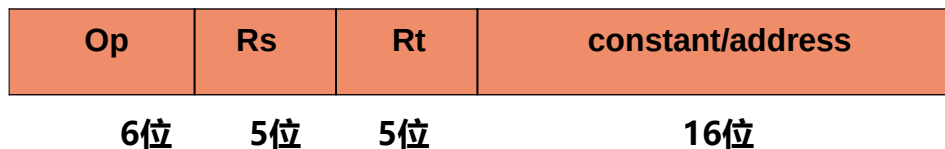
指令	op	rs	rt	rd	shamt	func
add	0	寄存器	寄存器	寄存器	0	32
sub	0	寄存器	寄存器	寄存器	0	34

- add \$t0, \$t3, \$s1 (含义为 $\$t0 = \$t3 + \$s1$) 中，\$t0编号为8，\$t3编号为11，\$s1编号为17，因此可以翻译为

	op	rs	rt	rd	shamt	func
	6位	5位	5位	5位	5位	6位
十进制	0	11 $\$t_3$	17 $\$s_1$	8 $\$t_0$	0	32
二进制	000000	01011	10001	01000	00000	100000

I (IMMEDIATE)类型指令

- I 型指令：将rd, shamt和func三个域合并为一个16位的新域constant/address, 存放一个16位的立即数或地址偏移量
 - constant/address字段表示的立即数的范围： $-2^{15} \sim 2^{15}-1$ ，即-32768~32767
- 典型的I型指令是访问内存中的数据，分支语句和立即数运算
 - MIPS中能访问内存的指令只有lw和sw等少数几条，其他指令的操作数都要求必须是寄存器或立即数



I (IMMEDIATE)类型指令

- 代表性的 I 型指令包括addi、lw和sw。
 - addi的功能是寄存器中的数字和立即数相加；
 - lw全称为load word，即从内存中加载一个字（32位）到寄存器中；
 - sw全称位store word，即将寄存器中的一个字存储到内存中

指令	op	rs	rt	constant/address
lw	35	寄存器	寄存器	地址偏移量
sw	43	寄存器	寄存器	地址偏移量
addi	8	寄存器	寄存器	常数

I (IMMEDIATE) 类型指令-偏移地址

- 在32位计算机中，地址长度为32位，共有 2^{32} 个有效地址
- MIPS是以字节为单位进行编址，因此MIPS 32最多支持 2^{32} 个字节，即能访问的最大存储空间为4GB，也就是 2^{30} 个字
- I型指令的address字段只有16位，用来表示地址的偏移量，需要配合一个存储基地址的寄存器来表示真正要访问的内存地址。

I (IMMEDIATE) 类型指令

- 示例:

假设变量x存在\$s0中，以字为基本单位的数组Array的基地址放在\$s1中，C语言语句 $\text{Array}[100] = x - \text{Array}[12]$ 翻译为MIPS汇编语言，需要用下面几条指令实现

```
lw      $t0, 48($s1)  $t0 = Array[12]
sub      $t0, $s0, $t0  ($t0 = $s0 - $t0)  x - Array[12]
sw      $t0, 400($s1)  400/4
```

I 型指令&汇编代码示例

- 汇编语句lw \$t0, 48(\$s1)可以按照下图方式翻译为机器指令

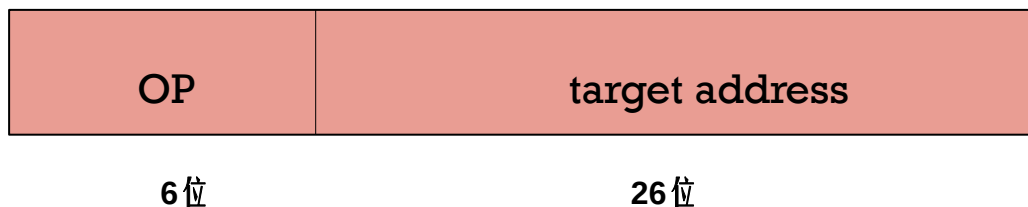
	op	rs	rt	immediate/address
	6位	5位	5位	16位
十进制	35	17 <i>\$s1</i>	8 <i>\$t0</i>	48
二进制	100011	10001	01000	0000000000110000

- 汇编语句sw \$t0, 400(\$s1)可以按照下图方式翻译为机器指令

	op	rs	rt	immediate/address
	6位	5位	5位	16位
十进制	43	17 <i>\$s1</i>	8 <i>\$t0</i>	400
二进制	101011	10001	01000	0000000110010000

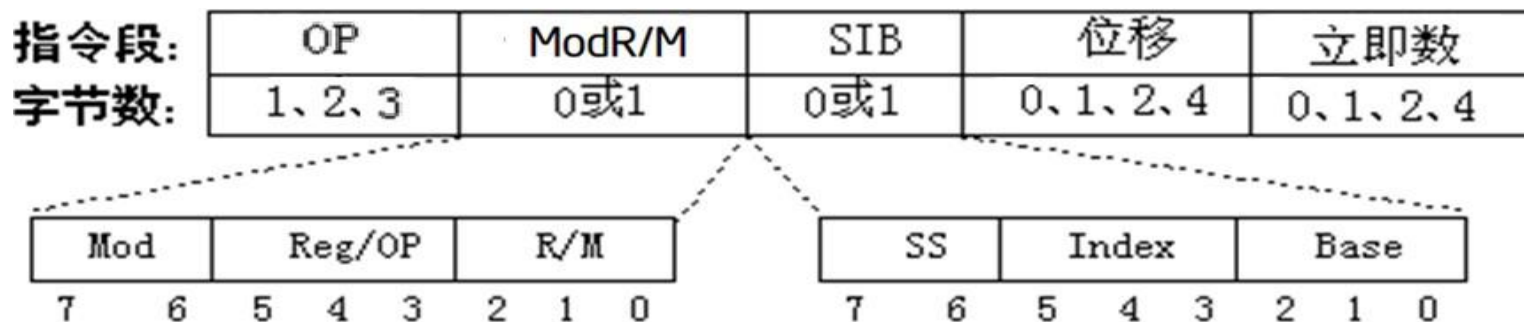
J (JUMP)类型指令

- J 型指令数量上最少，主要用于跳转语句，因此需要一个较长的字段存储跳转的目标地址
- J 型指令中除了6位的op字段外，其余的26位都用来存放一个立即数，即跳转的目标地址 (target address)



IA-32的指令格式

- 指令字长度：变长指令字，长1~15字节
- 操作码长度：变长操作码编码，长1~3字节
- 一条指令由两部分组成：前缀段+指令段，其中前缀段可以没有



主要内容

■ 3.1 汇编语言和指令系统的发展

■ 3.2 指令格式

■ **3.3 寻址方式**

■ 3.4 指令功能/类型

■ 3.5 MIPS汇编程序

3.3 寻址方式

- 寻址方式（或编址方式）
 - 确定一条指令的数据地址及下一条要执行的指令地址的方法
 - 与计算机硬件结构紧密相关，对指令格式和功能有很大影响
- 不同的寻址方式，其基本原理是相同的
 - 有的计算机寻址种类较少，因此在指令的操作码中表示出寻址方式；
 - 有的计算机采用多种寻址方式，此时在指令中专设一个字段表示一个操作数的来源或去向，如X86指令

常见的寻址方式

- **立即数寻址**
- **寄存器寻址**
- **直接寻址**
- **间接寻址**：寄存器间接寻址，存储器间接寻址
- **偏移寻址**：相对寻址，基址寻址，变址寻址

立即数寻址

- 立即数（或直接数）寻址方式：所需的操作数由指令的地址码部分直接给出。

- 例：MOV AX, 1000H

- 特点

- 执行速度快，操作码和操作数同时被取出，不必再次访问存储器。
- 操作数的大小受地址字段长度的限制

寄存器寻址

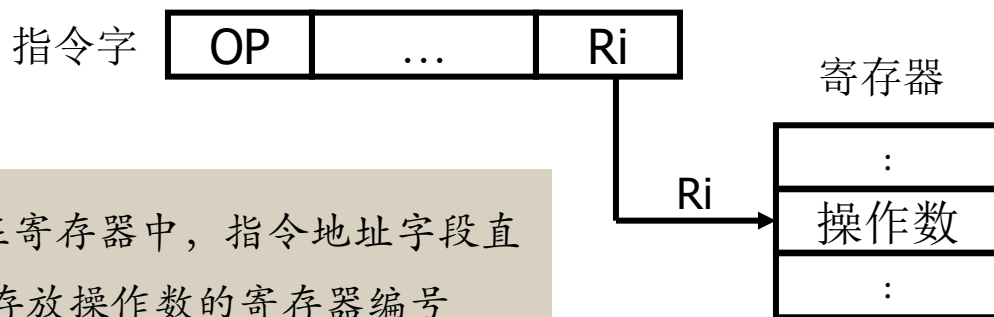
■ 寄存器寻址：

- CPU一般设置有一定数量的通用寄存器，用以存放操作数或中间结果。
- 指令地址码部分给出某一通用寄存器地址，所需的操作数就在这一寄存器中。

■ 特点

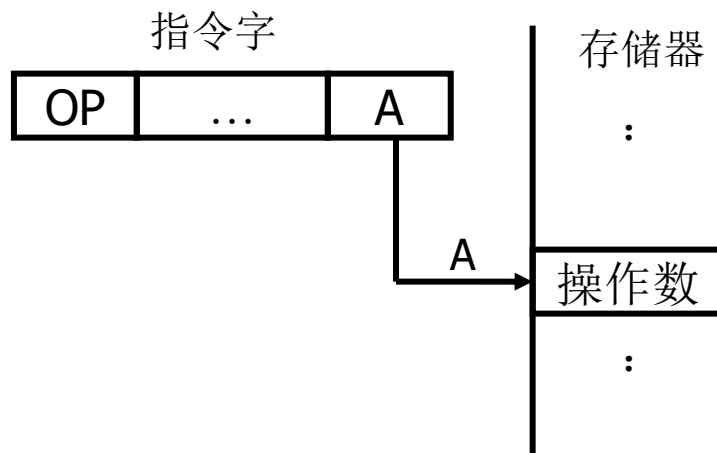
- 地址码短：通用寄存器的数量一般在几个至几十个之间，比存储单元少很多。
- 无需访存，指令执行速度快：从寄存器中存取数据比从存储器中存取快得多。
- 使用最多，是提高性能的常用手段。

例：MOV BX, AX



直接寻址

指令的地址码部分为操作数在存储器中的地址。



例：MOV AX, [1000H]

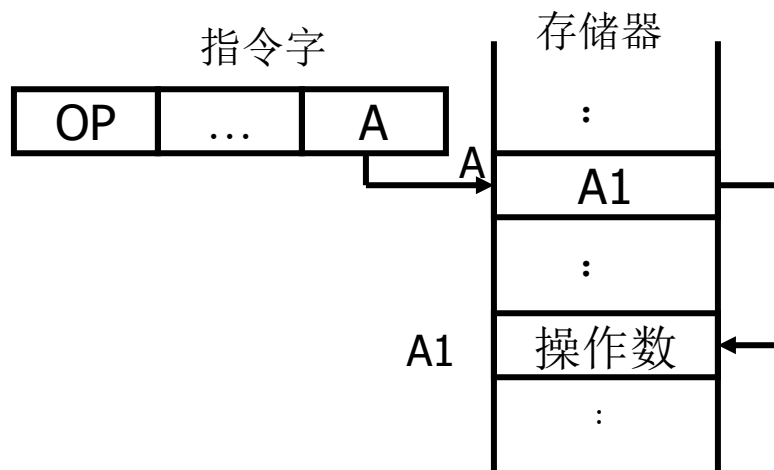
特点：

1. 处理简单、直接。
2. 寻址空间受到指令的地址字段长度限制。
3. 较少使用，x86-32位支持，x86-64无法直接访问高地址。

间接寻址（存储器间接寻址）

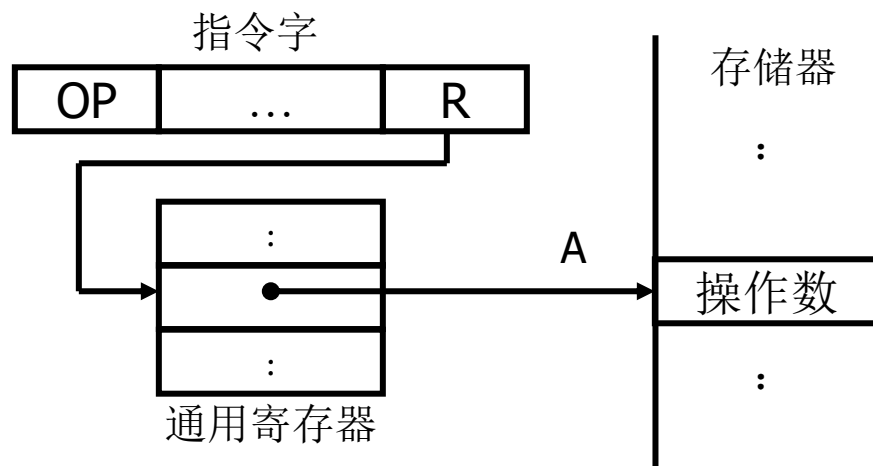
间接寻址：可分为寄存器间接寻址和存储器间接寻址两种方式。

- **存储器间接寻址**
 - 指令地址字段给出的存储器地址 A 存放的是操作数的地址
 - 操作数和操作数地址都在存储器中
- **特点**
 - 寻址空间大，灵活，便于编程
 - 至少需要两次访存才能取到操作数，执行速度慢，现在基本不用



间接寻址（寄存器间接寻址）

- 寄存器间接寻址
 - 指令地址字段给出的寄存器存放的是操作数在存储器中的地址
 - 操作数在存储器中
 - 操作数地址在寄存器中
- 特点
 - 比存储器间接寻址少访问存储器一次
 - 寻址空间大，使用比较普遍



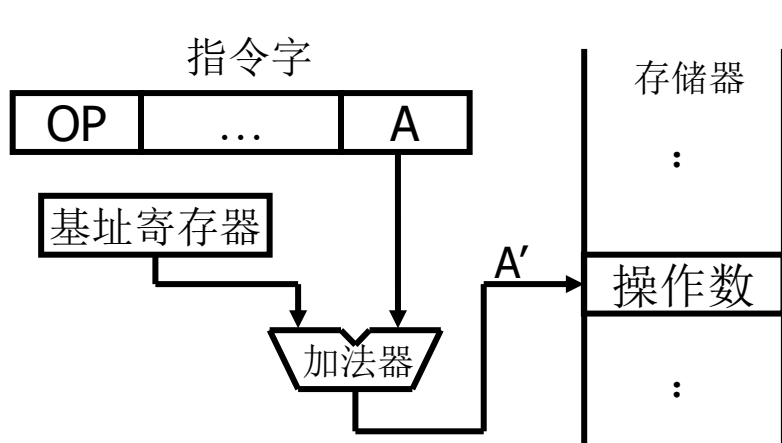
PC相对寻址

- 把程序计数器PC的内容（即当前执行指令的地址）与指令的地址码部分给出的位移量（disp）之和作为操作数的地址或转移地址，称为PC相对寻址。
- 主要用于转移指令，执行本条指令后，将转移到 $(PC) + \text{disp}$, (PC)为程序计数器的内容。
- PC相对寻址的两个特点：
 1. 转移地址不是固定的，随着PC值的变化而变化，并且总是与PC相差一个固定值，因此无论程序装入存储器的任何地方，均能正确运行，对浮动程序很适用。
 2. 位移量可正、可负，通常用补码表示。如果位移量为n位，则这种方式的寻址范围在 $(PC) - 2^{(n-1)} \sim (PC) + 2^{(n-1)} - 1$ 之间

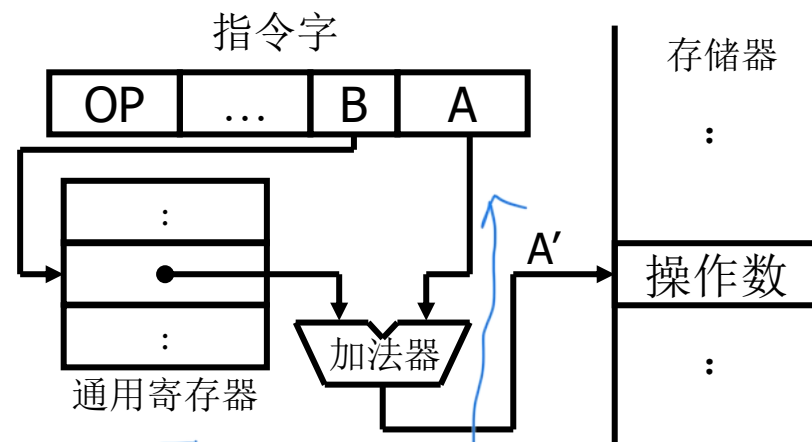
$$[-2^{n-1}, 2^{n-1} - 1]$$

基址寻址

基址寻址： 设置一个专用的基址寄存器，或由指令指定一个通用寄存器为基址寄存器，操作数的地址由基址寄存器的内容和指令的地址码A相加得到。



(a) 专用基址寄存器

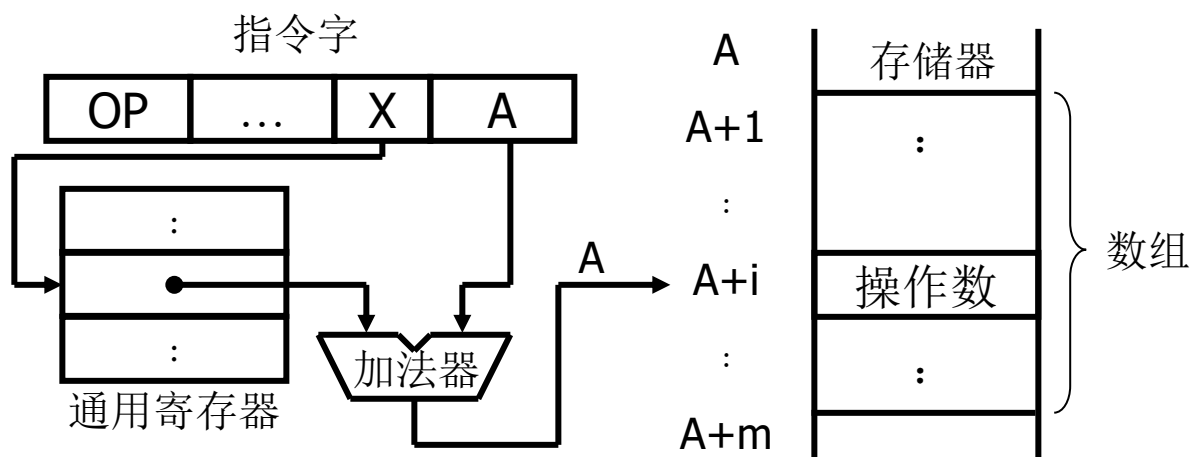


(b) 通用寄存器作为基址寄存器

例：MOV AX, 500H[BX]

变址寻址

指令地址码部分给出的地址A和指定的变址寄存器X的内容通过加法器相加，所得的和作为地址从存储器中读出所需的操作数。



基址寻址 & 变址寻址

基址寻址

◆ 对于一道程序，基址是**不变的**，程序中的**所有地址**都是相对于基址变化

◆ 在基址寻址中，**偏移量位数较短**

◆ 基址寻址立足于**面向系统**，主要是解决程序逻辑空间与存储器物理空间的无关性

变址寻址

◆ 对于变址寻址，形式地址给出的是一个存储器地址基准，变址寄存器存放的是相对于该基准地址的偏移量

◆ 在变址寻址中，偏移量位数足以表示**整个存储空间**

◆ 变址寻址立足于用户，主要是为编写**高效访问一片存储空间**的程序

类型	计算公式	典型应用场景
基址寻址	$EA = Base + A$	多进程地址空间隔离
变址寻址	$EA = A + Index$	数组循环访问
相对寻址	$EA = PC + A$	条件跳转指令

复合寻址

- 复合寻址：组合使用前面介绍的寻址方式。
 - 例如在一条指令中可以同时实现基址寻址与变址寻址，其有效地址为：基址寄存器内容+变址寄存器内容+指令地址码A
`mov eax, [ebx+esi*4+0x10]`：如果EBX指向一个结构体数组的起始位置，ESI用作数组索引，那么这条指令就可以用来从数组中读取特定元素
- 不同计算机采用的寻址方式是不同的，即使是同一种寻址方式，在不同的计算机中也有不同的表达方式或含义。
 - 用高级语言编程，不用考虑寻址方式，由编译程序负责；
 - 若用汇编语言编程，则应对寻址方式有确切的了解，才能编出正确而又高效率的程序。

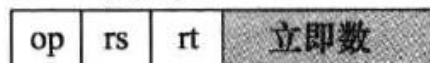
MIPS中用到的寻址方式

- 寄存器寻址 (register addressing)：直接以寄存器中的值作为操作数
- 基址寻址 (base addressing)：操作数在内存中，其地址是指令中指定的基址寄存器和地址偏移量（立即数）的和，例如：lw \$t1, 8(\$sp)
- 立即数寻址 (immediate addressing)：操作数是I型指令中的16位立即数，例如addi指令中指定的立即数
- PC相对寻址 (PC-relative addressing)：I型指令中，操作数地址是程序计数器PC和指令中16位立即数（有符号数）的和，具体操作上需要先将16位立即数左移两位再与PC相加，例如beq分支指令
- 伪直接寻址 (pseudodirect addressing)：用于J型指令，即跳转指令，跳转的目标地址是指令中的26位地址域左移两位和PC计数器的高4位相连

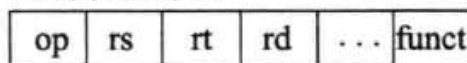
字节对齐

MIPS三种类型指令的寻址方式

1. 立即数寻址

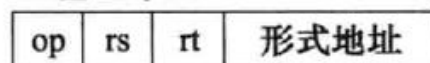


2. 寄存器寻址

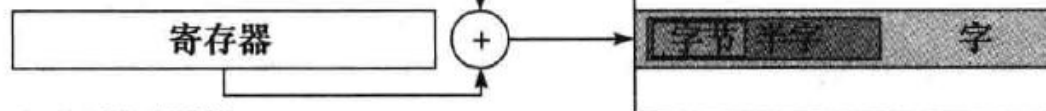


寄存器
寄存器

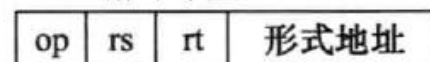
3. 基址寻址



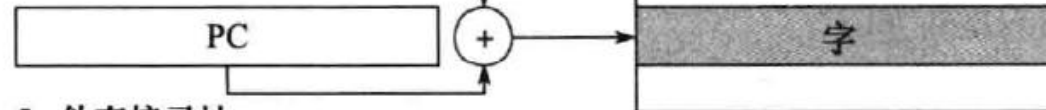
存储器



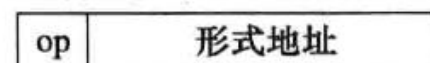
4. PC相对寻址



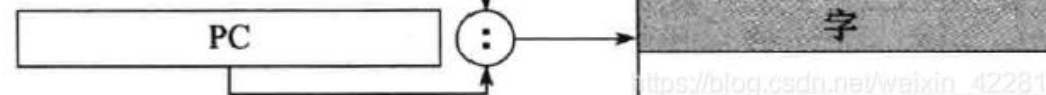
存储器



5. 伪直接寻址



存储器



https://blog.csdn.net/weixin_42281802

主要内容

■ 3.1 汇编语言和指令系统的发展

■ 3.2 指令格式

■ 3.3 寻址方式

■ **3.4 指令功能/类型**

■ 3.5 MIPS汇编程序

3.4. 指令功能/类型

指令系统：

- ◆决定了计算机的基本功能，不仅与计算机的硬件结构紧密相关，而且直接影响到编写操作系统和编写编译程序的难易程度，对机器的性价比有很大影响。
- ◆一台计算机最基本的、必不可少的指令并不多，很多指令都可以用这些最基本的指令组合来实现。
 - ◆例如，乘法（除法）运算指令、浮点运算指令，既可以直接用硬件实现，也可用其他指令编成子程序来实现；但两者在执行时间上差别很大。
 - ◆在指令系统中，有相当一部分指令是为了提高程序的执行速度和便于程序员编写程序而设置。
 - ◆指令系统的有效性还表现在用它所编制的程序占用的存储器空间小。

指令功能/类型

按指令所完成功能可分为：

- 算术逻辑运算指令
- 移位操作指令
- 浮点运算指令
- 十进制运算指令
- 字符串处理指令
- 数据传送指令
- 转移指令
- 堆栈操作指令
- 输入输出指令
- 特权指令

3.4.1 算术逻辑运算指令

■ 算术运算指令：

- 支持二进制加、减法、比较和求补码（取负数）等最基本的指令，以及乘、除法运算指令。
 - 这里讲的算术运算指的是定点数运算，即相当于高级语言中对整数的处理。
 - X86, Arm架构根据算术运算的结果可以置状态寄存器的状态位。
 - 如Arm结构有Z（为0）、N（为负）、V（溢出）、C（产生进位或借位）等状态位；x86 架构中有 **OF**（溢出）、**SF**（符号位）、**ZF**（为0）、**CF**（进位）

■ 逻辑运算指令：

- 对两个数进行与、或、非（求反）、异或（按位加）等操作。
- 一些计算机还设置有位操作指令，如位测试（测试指定位的值）、位清除（把指定位清零）、位求反（取某位的反值）指令等。

MIPS中的算术运算指令

- MIPS中算术运算指令:
 - **add** 和 **sub**: 寄存器间进行加法和减法操作,
 - 如 `add $s0,$s1,$s2`
`sub $s3,$s4,$s5`
 - **addi**: 是寄存器和立即数之间的相加, 如 `addi $s0,$s1,10` 。
 - MIPS中没有支持立即数的减法指令, 立即数用补码表示, 可以表示负数, 与负数的加法就相当于减法, 因此没有必要设置专门的立即数减法指令。

4位二进制数在不同表示方法下代表的数值

二进制数	无符号数	原码	补码	反码	移码
0000	0	+0	+0	+0	-8
0001	1	+1	+1	+1	-7
0010	2	+2	+2	+2	-6
0011	3	+3	+3	+3	-5
0100	4	+4	+4	+4	-4
0101	5	+5	+5	+5	-3
0110	6	+6	+6	+6	-2
0111	7	+7	+7	+7	-1
1000	8	-0	-8	-7	+0
1001	9	-1	-7	-6	+1
1010	10	-2	-6	-5	+2
1011	11	-3	-5	-4	+3
1100	12	-4	-4	-3	+4
1101	13	-5	-3	-2	+5
1110	14	-6	-2	-1	+6
1111	15	-7	-1	-0	+7

I 型指令的问题

◆ 立即数太大怎么办？

- 立即数在其字段内可以表示的时候，lw, sw 指令执行时都没有问题
- 立即数太大的解决方案
 - 使用软件技巧 + 新的指令
 - 不改变现有指令：只要加入一条新指令来帮忙

◆ 新指令：lui register, immediate

- Load Upper Immediate: 将一个16-bit的立即数存入寄存器的高16位，将寄存器的低16位全部置0

例：addi \$t0,\$t0, 0xABABCD CD

改为：lui \$at, 0xABAB
ori \$at, \$at, 0xCDCD
add \$t0,\$t0,\$at

这条指令根本放不进去

每条I-format指令只有16-bit用来存放立即数

MIPS中的逻辑运算指令

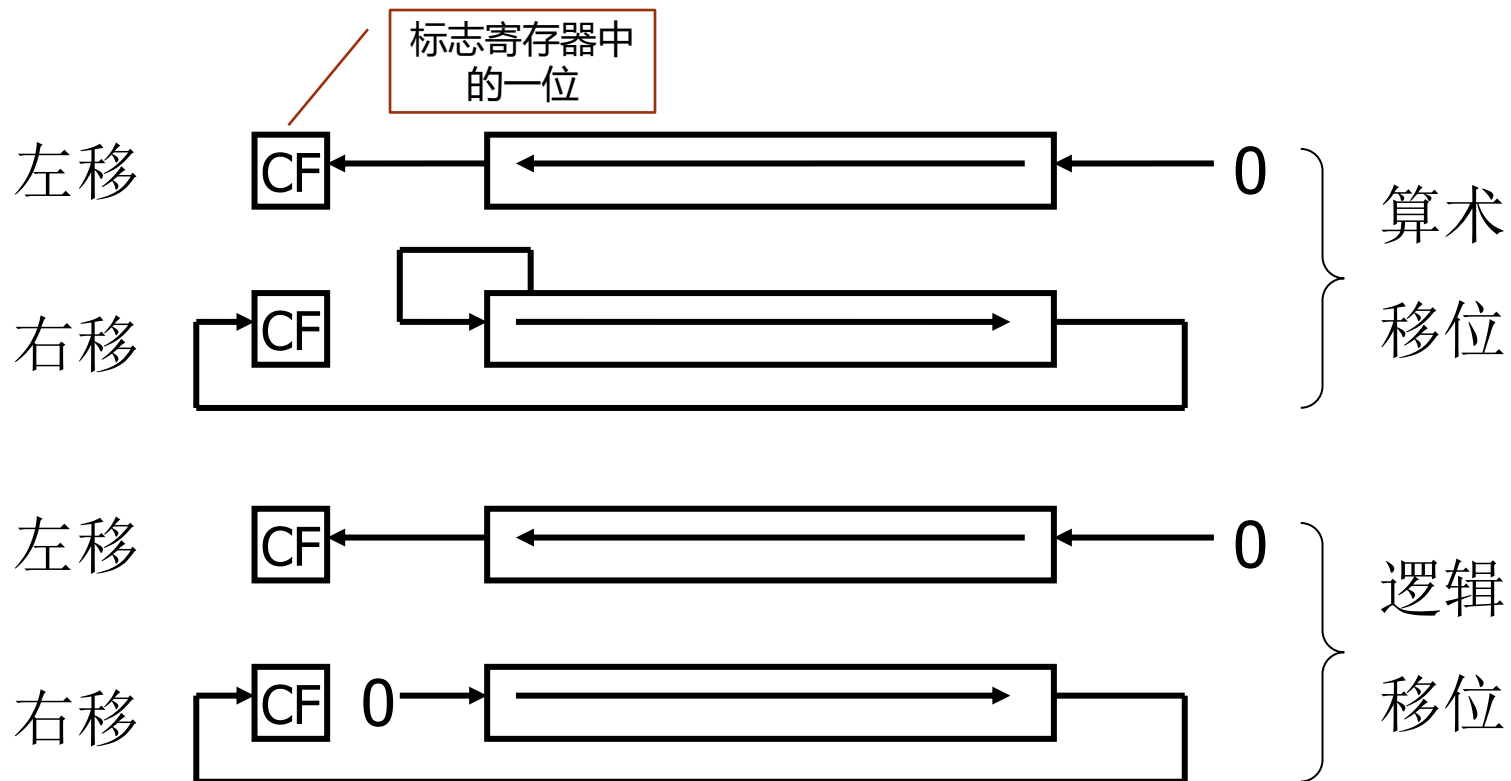
- MIPS中典型的逻辑运算
 - **and**: 按位与操作
 - **or**: 按位或操作
 - **nor**: 或非操作, 为了保持三操作数的格式用来取代not, 因为 $A \text{ nor } 0 = \text{not} (A \text{ or } 0) = \text{not } A$
 - **andi**和**ori**: 也是逻辑与/或功能, 只是源操作数有一个是立即数

3.4.2 移位操作指令

移位操作指令分为算术移位、逻辑移位和循环移位三种。

- 逻辑移位移位规则：
 - 逻辑左移时，高位移丢，低位添0
 - 逻辑右移时，低位移丢，高位添0
- 算术移位与逻辑移位类似，差别在于
 - **操作对象不同**：算术移位的操作数为带符号数，逻辑移位的操作数为无符号数
 - **右移移位**操作填入最高位的数据不同：算术右移保持最高位（符号位）不变，而逻辑右移最高位补零。

移位操作图示 (X86)



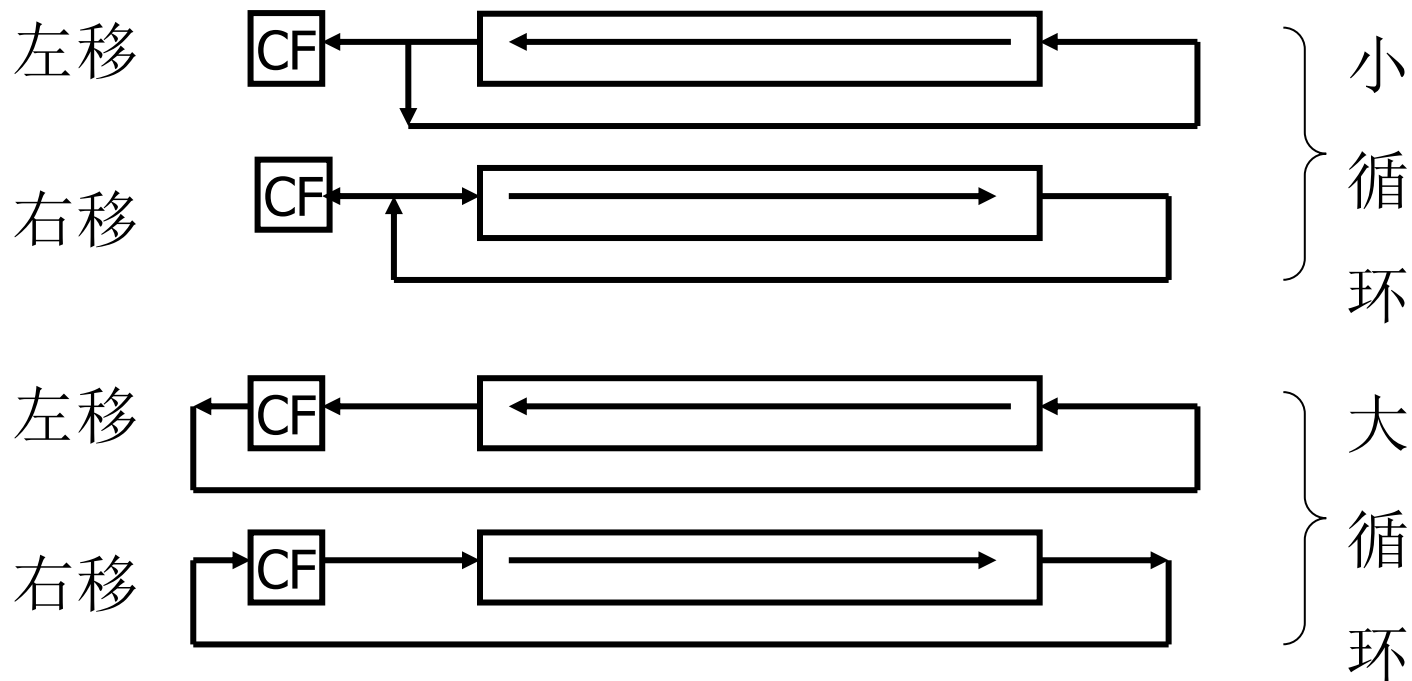
原数: 10011001 (二进制) = -103 (十进制)
ASR 2: 11100110 (二进制) = -26 (十进制)
LSR 2: 00100110 (二进制) = 38 (十进制)

移位操作指令 (X86)

■ 循环移位

- 按是否与“进位”位CF一起循环，分为小循环（即自身循环）和大循环（即和进位位CF一起循环）两种。
- **不带进位的循环右移**：操作数循环右移位，空出的高位部分由移出的低位部分来填充。同时,位CF中保存最后一次移出的那一位的内容。
- **带进位的循环右移**：操作数连同位CF循环右移，用原位CF的值填补空出的位，移出的位再进入位CF。
- **不带进位的循环左移**：操作数循环左移位，空出的低位部分由移出的高位部分来填充。同时,位CF中保存最后一次移出的那一位的内容。
- **带进位的循环左移**：操作数连同位CF循环左移，用原位CF的值填补空出的位，移出的位再进入位CF。

移位操作指令图示 (X86)



MIPS中的移位指令

- MIPS中的逻辑移位指令
 - `sll`: 逻辑左移, 例如 `sll $t0, $t1, 4`, 即将寄存器 `$t1` 中的内容逻辑左移 4 位, 即相当于乘以 2^4
 - `srl`: 逻辑右移。
- MIPS中的算术移位指令
 - `sra`: 右移并作符号扩展填充
 - 算术左移与逻辑左移一样

3.4.3 浮点运算指令

- 高级语言中的实数（real）经常是先转换成浮点数的形式而后再进行处理。
- 浮点运算指令，一般能对单精度（32位）、双精度（64位）数据进行处理。
- FPU: float-point unit. 浮点处理单元，用来进行浮点运算。

3.4.4 十进制运算指令

- 动机：
 - 人机交互作用时，输入输出的数据都是以十进制形式表示的。
 - 设置十进制运算指令用硬件提高数据处理的速度。
 - 若无十进制运算指令，需要首先将十进制数据转换成二进制数，再在机器内运算；然后又转换成十进制数据输出。
 - 十进制的调制指令：DAA, DAS, AAA, AAS, AAM, AAD
- 现状：微码实现，在现代CPU上运行慢，X86-64基本不用

3.4.5 字符串处理指令

- 字符串处理指令：非数值处理指令，一般包括字符串传送、字符串比较、字符串查询、字符串转换等指令。
 - 字符串传送：数据块从主存储器的某区传送到另一区域，如movs；
 - 字符串比较：一个字符串与另一个字符串逐个字符进行比较，以确定其是否相等，如cmps；
 - 字符串查询：查找在字符串中是否含有某一指定的子串或字符；
 - 字符串转换：从一种数据表达形式转换成另一种表达形式。例如，从ASCII码转换成EBCDIC码（扩充的BCD码）。这种指令在需对大量字符串进行各种处理的文字编辑和排版方面非常有用。

3.4.6 数据传送指令

- 数据传送指令：实现寄存器与寄存器，寄存器与存储器单元，存储器单元与存储器单元之间的数据传送。
 - 对于存储器来讲，数据传送包括了对数据的读（相当于取数指令）或写（相当于存数指令）操作。
 - 数据传送时，数据从源地址传送到目的地址，而源地址中的数据保持不变，因此实际上是数据复制。
 - 数据传送指令一次可以传送一个数据或一批数据。
 - Intel8086的MOVS指令，一次传送一个字或字节，加上重复执行前缀（REP）后，x86-32架构一次可以传送多达4GB的数据块。

MIPS中的数据传送指令

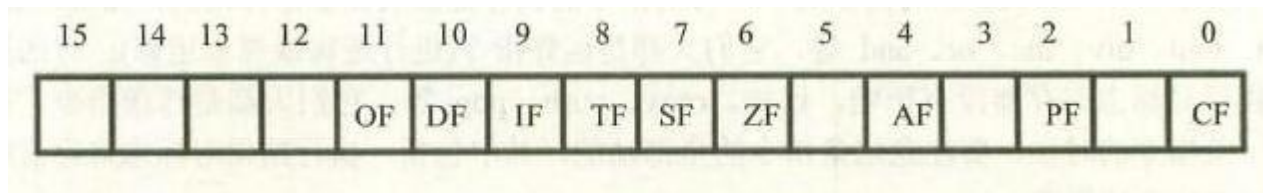
- MIPS数据传送指令：在内存与寄存器之间传输数据
- 内存到寄存器: **lw** (取字) , **lb** (取字节) , **lh** (取半字)
 - 指令语法：操作码, 寄存器, 数值偏移量(寄存器)
 - 例: `lw $t0, 12($s0)`; `$s0` 称为基址寄存器, 12 称为偏移量
 - 例: `lb $s0, 3($s1)`; 把内存中的某个地址(“3” + `s1`中的地址值)所存储的数值拷贝到`s0`的低地址字节上, 并使用位扩展(或称为符号扩展)填充32位中剩余的24位。
- 寄存器到内存: **sw** (存字) , **sb** (存字节) , **sh** (存半字)
 - 指令语法：操作码, 寄存器, 数值偏移量(寄存器)
 - 例: `sw $t0, 12($s0)`
- 字对齐 (Alignment): 对象的起始地址一定要是字长的整数倍
- 无论是 `lw` 还是 `sw`, 基址寄存器的值与偏移量的和始终应该是4的倍数

3.4.7 转移类指令

- 转移指令用以控制程序流的转移。
 - 大多数情况下，计算机按顺序执行程序，但是也经常可能会离开原来的顺序转移到另一段程序或循环执行某段程序。
- 按转移的性质，转移指令分为无条件转移、条件转移、过程调用与返回、陷阱（trap）等几种。

无条件转移与条件转移

- 无条件转移指令：不受条件约束，直接把程序转移到指令所规定的目的地。
- 条件转移指令：测试设置的转移条件，如果满足转移条件，则转到目标地址所指示的指令执行，否则继续执行下一条指令。
 - 8086状态寄存器，标志位由算术和逻辑指令（如 ADD, SUB, CMP, AND 等）自动设置。



进位标志CF，奇偶标志PF，辅助进位标志AF，零标志ZF，符号标志SF，自陷标志TF，中断标志IF，方向标志DF，溢出标志OF

- x86-64 的条件转移指令（如 JE, JNE, JL, JG, JB 等）根据 RFLAGS 寄存器中的标志位状态来决定是否跳转。

MIPS中的转移指令

- 无条件转移:

- `J label`; 跳转到标签label所在的代码, 不需要满足任何条件
- 相当于C当中: `goto label`

- 条件转移:

- `beq reg1, reg2, Label`: 当reg1和reg2中的值相等时, 程序将跳转到Label标签所在的代码行继续执行
 - 相当于C当中: `if (register1==register2) goto Lable`
- `bne reg1, reg2, Label`: 当reg1和reg2中的值不相等时, 程序将跳转到Label标签处
 - 相当于C当中: `if (register1!=register2) goto Lable`

MIPS中的转移指令示例

1. 无条件转移, 如

j Skip

add \$t0, \$t1, \$t2

Skip: ...

上述代码中

add \$t0, \$t1, \$t2语

言永远不会被执行

2. 有条件转移

int sum = 0;

if (a == b)

sum -= a;

else

sum += a;



假设变量a存储于\$t0, 变量b存储于\$t1, 变量sum存储于\$t2

move \$t2, \$zero

bne \$t0, \$t1, Else

sub \$t2, \$t2, \$t0

j OK

Else: add \$t2, \$t2, \$t0

OK: ...

其中\$zero是MIPS中第0个寄存器, 其值固定为0

MIPS中的转移指令

- 如果需要根据大于、小于等关系跳转到不同的分支，或者完成循环操作，则需要用到slt和slti指令。
 - `slt $t0, $s3, $s4` *set less than*
 - 这条指令判断寄存器\$s3和\$s4的大小，如果 $\$s3 < \$s4$ ，则\$t0被设置为1，否则\$t0被设置为0。
 - `slti $t0, $s1, 12`
 - 这条指令会判断寄存器\$s1和立即数12之间的大小关系，如果 $\$s1 < 12$ ，则\$t0被设置为1，否则\$t0被设置为0。

MIPS中的转移指令

while (a >= 10) a++;

假设变量a存储于寄存器\$t0中，则可以翻译为下列汇编语句

StartWhile: slti \$t1, \$t0, 10

 bne \$t1, \$zero, EndWhile

 addi \$t0, \$t0, 1

 j StartWhile

EndWhile: ...

MIPS中的转移指令

- 利用slt、beq和bne的组合，可以根据两个寄存器之间各种大小关系（等于、不等于、小于、大于、小于等于、大于等于）决定是否跳转

\$s0 vs. \$s1	汇编语句
==	beq \$s0, \$s1, Label
!=	bne \$s0, \$s1, Label
<	slt \$t0, \$s0, \$s1 # 若\$s0<\$s1,\$t0被设置为1 bne \$t0, \$zero, Label #不等于0，因此跳转
>	slt \$t0, \$s1, \$s0 # 若\$s1<\$s0, \$t0被设置为1 bne \$t0, \$zero, Label #不等于0，因此跳转
<=	slt \$t0, \$s1, \$s0 #若\$s1>\$s0, \$t0被设置为0 beq \$t0, \$zero, Label # 等于0，因此跳转
>=	slt \$t0, \$s0, \$s1 #若\$s0<\$s1, \$t0被设置为0 beq \$t0, \$zero, Label # 等于0，因此跳转

调用指令与返回指令

- 调用指令：包括过程调用、系统调用、转子程序，实现从一个程序转移到另一个程序的操作。
- 调用指令需要保留返回地址，也就是说当执行完被调用的程序后要回到原调用程序，继续执行过程调用指令的下一条指令。
- 返回地址一般保留于堆栈中，随同保留的还有一些状态寄存器或通用寄存器内容。
- 调用指令与返回指令是一对配合使用的指令，返回指令从堆栈中取出返回地址，继续执行调用指令的下一条指令。

MIPS中的调用指令和返回指令

- MIPS中为了完成和加速子过程调用，设置了一些**专用的寄存器**：
 - **\$a0~\$a3**：四个参数寄存器，用于传递参数；
 - **\$v0~\$v1**：两个返回值寄存器，用于存储子过程的返回值；
 - **\$ra**：一个返回值寄存器，用于存储主过程中的返回地址。

名称	编号	功能
zero	0	常数0
at	1	为汇编程序保留
v0-v1	2-3	过程调用返回值
a0- a3	4-7	过程调用参数
t0-t7	8-15	临时变量
s0-s7	16-23	保存
t8- t9	24-25	其他临时变量
k0-k1	26-27	为OS保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS中的调用指令和返回指令

- MIPS中的调用语句：jal，即跳转-链接指令
 - 格式为jal ProcedureAddress。
 - 这条指令首先将返回地址链接存储到寄存器\$ra中，然后转移到子过程的开始地址ProcedureAddress处继续执行。
- 子过程执行结束后，需要调用jr \$ra语句来返回主过程的返回地址处继续执行

陷阱 (TRAP) 指令

- 程序出现故障，或需要执行特权指令，计算机就发出陷阱信号，并暂停当前程序的执行（称为**中断**），转入故障处理程序进行相应的故障处理。
 - 硬中断：硬件故障时，由CPU自动产生并执行
 - 软中断：有些计算机设置可供用户使用的陷阱指令或“访管”指令，利用它来实现系统调用和程序请求

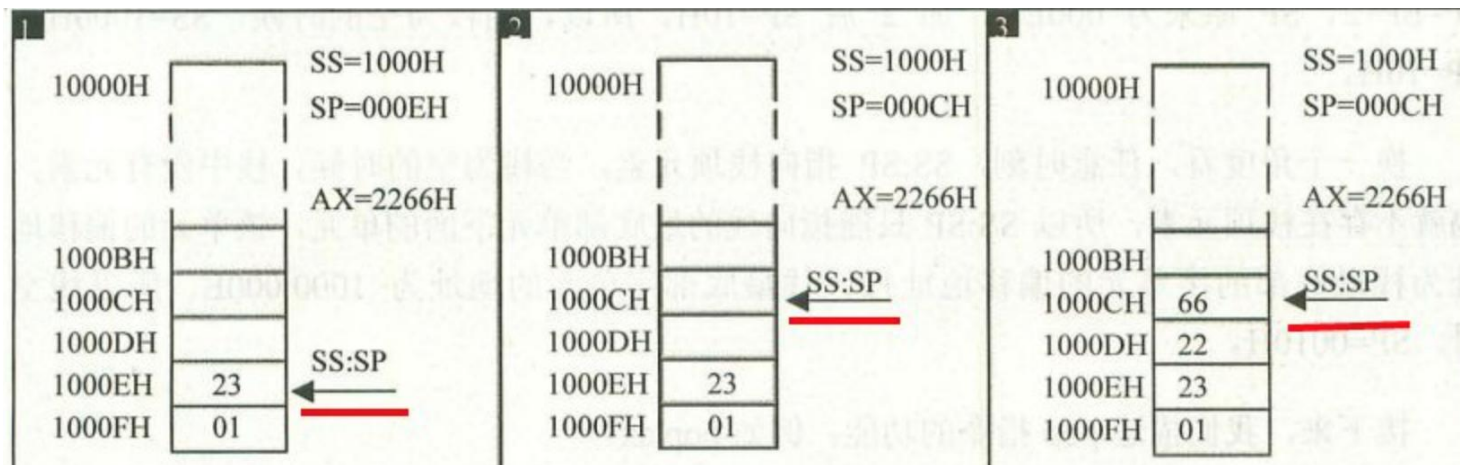
例如，x86的软件中断指令，用它可以完成系统调用过程：

INT n

CPU 会根据中断向量n 查中断描述符表 (IDT)，跳转到对应的系统调用处理程序。现代x86-64用syscall 取代 int 0x80进行系统调用。Mips也有syscall指令实现系统调用。

3.4.8 堆栈及堆栈操作指令

- 堆栈 (stack)：由若干个连续存储单元组成的先进后出 (FILO) 存储区
- 第一个送入堆栈中的数据存放在栈底，最近送入堆栈中的数据存放在栈顶，栈顶用**堆栈指针** (stack pointer, 简称**SP**) 记录
- 栈底是固定不变的，而栈顶却是随着数据的入栈和出栈在不断变化，任何堆栈操作只能在栈顶进行。
 - X86架构的栈是从高地址向低地址方向长



SS: 堆栈段起始地址，加上堆栈的容量可以得出栈底的位置

堆栈及堆栈操作指令

- 栈操作指令：Push、Pop

- 压入指令：push

PUSH OPR 把OPR（长度为4个字节）压入堆栈

其操作是： $(SP)-4 \rightarrow SP$

$OPR \rightarrow (SP)$

- 弹出指令：pop

POP OPR 弹出一个数据（长度为4个字节）送OPR

其操作是： $((SP)) \rightarrow OPR$

$(SP)+4 \rightarrow SP$

其中， (SP) 表示堆栈指针的值； $((SP))$ 表示SP所指的栈顶的值。

堆栈及堆栈操作指令

- 堆栈的另一个作用：用于子程序调用时参数的传递。
 - 当一个过程4个参数寄存器和2个返回值寄存器不够用时，可以使用栈结构把寄存器内容换出到存储器。
 - 利用堆栈传递参数时，首先把所需传递的参数压入堆栈中，然后调用子程序。
 - \$sp: 栈指针寄存器，29号寄存器
 - 执行过程前，把寄存器旧值压入栈，过程执行完毕，返回前从栈中弹出数据恢复寄存器旧值。

3.4.9 输入输出 (I/O) 指令

- 输入输出指令的一般格式如下：



- OP：操作码，表示I/O指令
 - REG：寄存器名，用于指定与外部设备交换数据的寄存器
 - A：外部设备中的寄存器地址或设备码，其长度一般为8~16位，可以表示256~64K个设备寄存器（输入数据寄存器 / 输出数据寄存器 / 控制寄存器）。
- 输入指令：从A地址指定的外部设备寄存器中读入一个数据到REG寄存器中，如x86的 `in al, 0x60`；
 - 输出指令：REG寄存器中的数据送到A地址所指定的外部设备寄存器中，如 x86的 `out 0x20, al`。
 - 有些架构计算机采用外部设备与存储器统一编址的方法，把外部设备寄存器看成是存储器的某些单元，不再专设I/O指令，如MIPS, ARM, RISC-V。
 - I/O指令可能有特权要求。

3.4.10 特权指令

- 特权指令：只能在处理器的最高特权级别（通常是内核态）执行。如果用户态程序尝试执行这些指令，CPU 会触发一个异常（如 General Protection Fault），由操作系统处理，通常会导致程序被终止。
- 特权指令通常用于操作系统内核管理硬件、内存、中断等关键资源。
 - I/O 端口访问指令：in、out等指令
 - 控制寄存器访问指令：读写 CPU 的控制寄存器（如 CR0, CR3, CR4），控制分页、保护模式等。
 - 调试和性能监控寄存器：RDMSR / WRMSR，读写 Model Specific Registers (MSR)，用于性能监控、CPU 配置等
 - 中断和异常控制指令：int 等指令
 - 任务和系统控制指令：halt停机指令

主要内容

■ 3.1 汇编语言和指令系统的发展

■ 3.2 指令格式

■ 3.3 寻址方式

■ 3.4 指令功能/类型

■ **3.5 MIPS汇编程序**

3.5 MIPS汇编程序

- 简介
- 指令集总结及使用
- MIPS汇编程序结构

3.5.1 简介

- MIPS: Microprocessor without Interlocked Piped Stages architecture
 - 精简指令集(RISC)的哲学, 1980年代提出
 - MIPS 简单、优雅, 不被细节所累
 - 广泛应用于嵌入式系统、网络设备、路由器、游戏机等领域。
 - 其设计理念深刻影响了 ARM、RISC-V 等后续架构。

3.5.2 MIPS指令集总结及使用

- 算术、逻辑和移位指令：只能操作寄存器, 不能直接操作内存
 - add, addu, addi, sub, subu, and, or, nor, xor, sll, srl, sra, slt, sltu
- 存/取指令：在内存与寄存器之间传输数据
 - lw, lb, lh, lbu, lhu, lui, sw, sb,sh
- 条件分支指令
 - beq, bne, slt, sltu, slti, sltiu
- 跳转指令
 - j, jal, jr

循环

考察C中循环的例子；假定A[] 是整型数组

```
do {  
  g = g + A[i];  
  i = i + j;  
} while (i != h);
```

重写成：

```
Loop: g = g + A[i];  
      i = i + j;  
      if (i != h) goto Loop;
```

假定g, h, i, j和A[]的基址分别对应\$s1, \$s2, \$s3, \$s4, \$s5
对应的MIPS 代码：

```
Loop: sll $t1, $s3, 2    # $t1 = 4*i  
      add $t1, $t1, $s5 # $t1 = addr A  
      lw  $t1, 0($t1)   # $t1 = A[i]  
      add $s1, $s1, $t1 # g = g + A[i]  
      add $s3, $s3, $s4 # i = i + j  
      bne $s3, $s2, Loop # goto Loop if i != h
```

C语言中有3种循环结构，While；do... while；for，每一种都可以用另外两种中的任一种等价表达，上面的例子同样适用于while 和 for 循环。

分支

根据k的取值从4个选项中选择1个，C代码如下：

```
switch (k) {  
case 0: f=i+j; break; /* k=0 */  
case 1: f=g+h; break; /* k=1 */  
case 2: f=g - h; break; /* k=2 */  
case 3: f=i - j; break; /* k=3 */  
}
```

先简化成if-else语句链

```
if(k==0) f=i+j;  
    else if(k==1) f=g+h;  
        else if(k==2) f=g - h;  
            else if(k==3) f=i - j;
```

变量存储的寄存器：

```
f:$s0,  
g:$s1,  
h:$s2,  
i:$s3,  
j:$s4,  
k:$s5
```

MIPS 代码如下

```
bne $s5,$0,L1    # branch k!=0  
add $s0,$s3,$s4 #k==0 so f=i+j  
j    Exit        # end of case so Exit  
L1: addi $t0,$s5,-1 # $t0=k-1  
    bne $t0,$0,L2    # branch k!=1  
    add $s0,$s1,$s2 #k==1 so f=g+h  
    j    Exit        # end of case so Exit  
L2: addi $t0,$s5,-2 # $t0=k-2  
    bne $t0,$0,L3    # branch k!=2  
    sub $s0,$s1,$s2 #k==2 so f=g-h  
    j    Exit        # end of case so Exit  
L3: addi $t0,$s5,-3 # $t0=k-3  
    bne $t0,$0,Exit # branch k!=3  
    sub $s0,$s3,$s4 #k==3 so f=i-j  
Exit:
```

函数调用

➤ 在函数调用中用到的指令：

- **jal 指令**：执行 'laj' (link and jump)，将返回地址存储在寄存器(\$ra)
 - 第一步 (link): 将下一条指令地址存入\$ra
 - 第二步 (jump): 跳转到给定的label
- **jr \$ra**：跳回\$ra中存储的地址

用寄存器来记录函数调用信息

寄存器规范

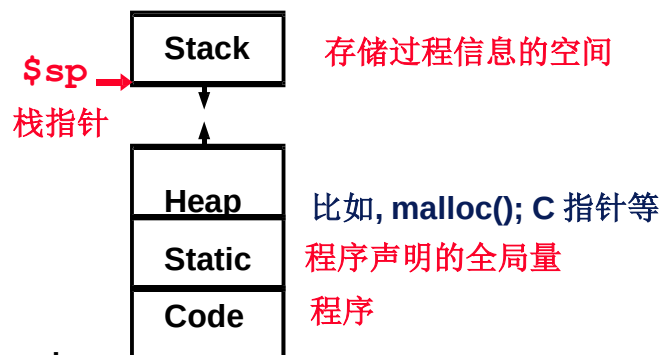
- 返回地址 **\$ra**
 - 参数 **\$a0, \$a1, \$a2, \$a3**
 - 返回值 **\$v0, \$v1**
 - 局部变量 **\$s0, \$s1, ... , \$s7**
- 还会用到栈！

```
main() {
    int i,j,k,m;
    ...
    i = mult(j,k); ...
    m = mult(i,i); ...
}

/* really dumb mult function */
int mult (int mcand, int mlier){
    int product;
    product = 0;
    while (mlier > 0) {
        product = product + mcand;
        mlier = mlier -1; }
    return product;
}
```

嵌套函数调用

```
int sumSquare(int x, int y) {  
    return mult(x,x)+ y;  
}
```



- ◆ 假设sumSquare被调用，而sumSquare又调用mult
 - \$ra中存储着 sumSquare 的返回地址，但是会在调用 mult时重写
 - 需要在调用mult 之前存储 sumSquare 返回地址
 - 嵌套调用需要在\$ra之外存储相关返回地址信息！
- ◆ 当一个C 程序运行时，有3块重要的内存区域被分配
 - 静态区 (static)：存储静态变量，一旦程序声明，直到程序执行结束才会清除，比如C程序的全局变量
 - 堆 (heap)：动态声明的变量
 - 栈 (stack)：程序执行过程中使用的空间，可用来存储寄存器值
- ◆ 嵌套调用使用栈
 - 寄存器\$sp指向栈顶
 - 压数据入栈的时候，对\$sp指针减去需要的空间量，然后向该空间填写信息

嵌套函数调用

```
int sumSquare(int x, int y) {  
    return mult(x,x)+ y;  
}
```

sumSquare:

“push”

```
addi $sp,$sp,-8 # space on stack  
sw $ra, 4($sp) # save ret addr  
sw $a1, 0($sp) # save y  
add $a1,$a0,$zero # prep args, $a0=x  
jal mult # call mult
```

“pop”

```
lw $a1, 0($sp) # restore y  
add $v0,$v0,$a1 # mult()+y  
lw $ra, 4($sp) # get ret addr #  
addi $sp,$sp,8 restore stack  
jr $ra
```

mult:

...

函数调用规则

◆ 调用的步骤

1. 将需要保存的值压入栈
2. 如果需要的话, 指定参数
3. jal 调用
4. 从栈中恢复相关的值

◆ 调用过程中的规则

- 通过 jal 指令调用, 使用一个 jr \$ra 指令返回
- 最多可接受4个入口参数, \$a0, \$a1, \$a2, \$a3
- 返回值通常在 \$v0 中(如果需要, 可以使用 \$v1)
- 必须遵守寄存器使用规范 (即使是在那些只有自己调用的函数中)

一个函数的基本结构

entry_label:

```
addi $sp,$sp,-framesize # space on stack
sw $ra, framesize-4($sp) # save $ra
..... # save other regs if need
```

```
jal** #call other functions
```

.....

Restore other regs if need

```
lw $ra, framesize-4($sp) # restore $ra
addi $sp,$sp, framesize # restore stack
jr $ra
```

函数调用的寄存器规则

◆ 保存寄存器

- \$0: 不能改变，永远是0
- \$s0-\$s7: 如果被修改了需要恢复。如果被调用函数由于各种原因改变了这些值，它必须在返回之前将这些寄存器的原始值恢复。
- \$sp: 如果被修改了需要恢复。栈指针在jal执行之前和之后必须是指向的同一个地址，不然调用函数无法从栈上正常恢复数据了。
- 所有保存寄存器都以S开头！

◆ 易变寄存器

- \$ra: 会改变。jal会自动更改这个寄存器值，调用函数需要将其值保存在栈上
- \$v0-\$v1: 会改变。始终保存最新的返回值
- \$a0-\$a3: 会改变。调用函数如果在调用完成后还要用到这些寄存器中的值，就要在调用前将这些值保存在自己的栈空间内
- \$t0-\$t9: 会改变。任何函数在任何时候都可以更新这些寄存器中的值，调用函数如果在调用完成后还要用到这些寄存器中的值，就要在调用前将这些值保存在自己的栈空间内

◆ 调用和被调用都只需要保存他们使用的临时和保存寄存器值，并非所有的。

MIPS汇编语言

◆ MIPS汇编中的3类语句

1. 可执行指令：为处理器生成在运行时执行的机器码，告诉处理器该做什么
2. 伪指令和宏：由汇编程序翻译成真正的指令，简化编程人员的工作
3. 汇编伪指令
 - 当翻译代码时为汇编程序提供信息，用来定义段、分配内存变量等
 - 不可执行: 汇编伪指令不是指令集的一部分

◆ 汇编语言指令格式

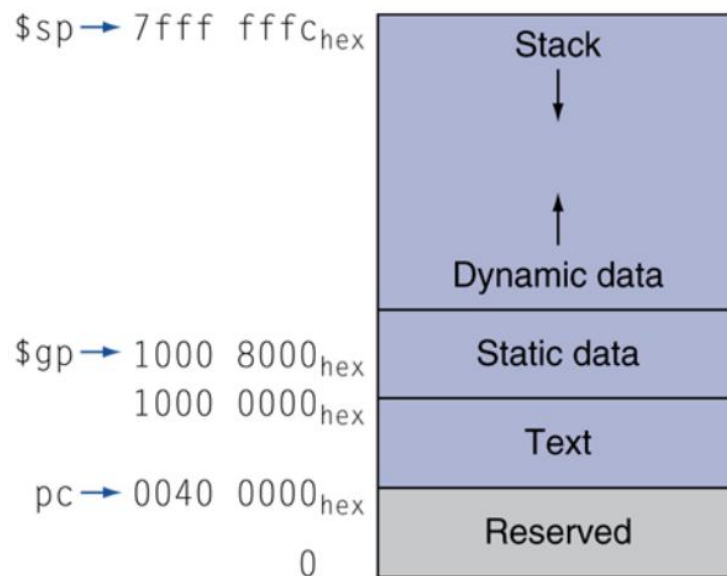
[标签:] 操作符 [操作数] [#注释]

- 标签: (可选)标记内存地址, 必须跟冒号; 通常在数据和代码段出现
- 操作符: 定义操作 (比如 add, sub, 等)
- 操作数
 - 指明操作需要的数据, 可以是寄存器、内存变量或常数
 - 大多数指令有3个操作数
- #注释: 由 '#' 开头在1行内结束; 非常重要!

3.5.3 MIPS汇编程序结构

■ 内存布局

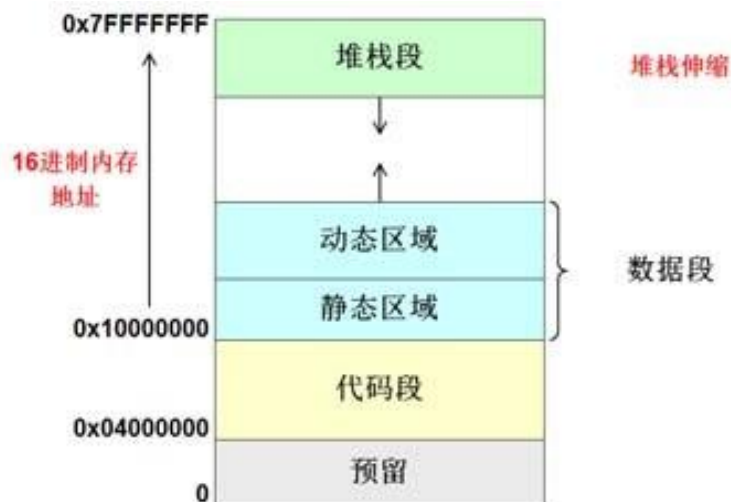
- Text: 程序代码段
- Static data: 全局变量和静态变量
 - 例如, C语言中的静态变量, 常数数组和串
 - \$gp 寄存器初始地址±偏移量寻址本段内存
- Dynamic data: 堆
 - 例如, C中的malloc, Java中的new
- Stack: 栈, 自动存储区



程序模板

- DATA, .TEXT, 和 .GLOBL 伪指令
 - .DATA 伪指令，定义程序的数据段，程序变量需要在该伪指令下定义
 - 汇编程序会分配和初始化变量的存储空间
 - .TEXT 伪指令，定义程序的代码段
 - .GLOBL 伪指令，声明一个符号为全局的变量或标签，可被其它文件引用
 - 用该伪指令声明一个程序的 main 过程

```
# Title:                               Filename:
# Author:                              Date:
# Description:
# Input:
# Output:
##### Data segment #####
.data
.
.
##### Code segment #####
.text
.globl main
main:                                # main program entry
.
.
li $v0, 10                          # Exit program
syscall
```



数据定义

◆ 为变量的存储划分内存

- 可能会有选择的为数据分配名字（标签）

◆ 语法

[名字] 伪指令 初始值[, 初始值] ...

var1: **.WORD** 10

- 所有的初始值在内存中以二进制数据存储

◆ 数据伪指令

- **.BYTE** 伪指令，以8位字节存储数值表
- **.HALF** 伪指令，以16位（半字长）存储数值表
- **.WORD** 伪指令，以32位（一个字长）存储数值表
- **.FLOAT** 伪指令，以单精度浮点数存储数值表
- **.DOUBLE** 伪指令，以双精度浮点数存储数值表

◆ 字符串伪指令

- **.ASCII** 伪指令，为一个ASCII字符串分配字节序列
- **.ASCIIZ** 伪指令，与**.ASCII** 伪指令类似，但字符串以NULL结尾
- **.SPACE *n*** 伪指令，为数据段中 *n* 个未初始化的字节分配空间
- 字符串中的特殊字符（按照C语言的约定），“新行: \n，Tab: \t，引用: \"

◆ MIPS中变量需内存对齐：数据的存储地址必须是其大小的整数倍。

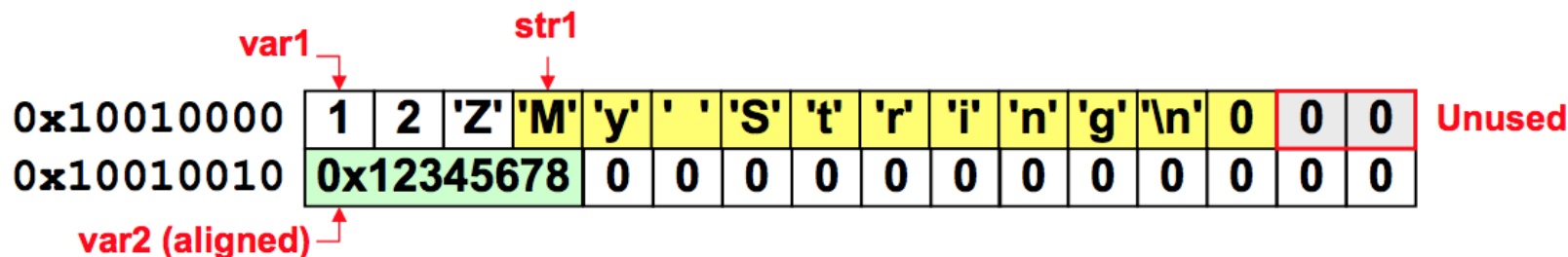
数据定义示例

```
.DATA
var1:  .BYTE    1, 2, 'Z'
str1:  .ASCIIZ  "My String\n"
var2:  .WORD    0x12345678
```

如果初始值超过了值域上界，
汇编程序会报错

- ❖ 汇编程序为标签（变量）构建符号表
 - 为数据段的每一个标签计算地址

标签	地址
var1	0x10010000
str1	0x10010003
var2	0x10010010



系统调用

- ◆ 程序通过系统调用实现输入/输出
- ◆ MIPS 提供特殊的 **syscall** 系统调用指令，从操作系统获取服务
 - 从 **\$v0** 寄存器中读取服务数
 - 从 **\$a0, \$a1**, 等寄存器中读取参数值（如果有）
 - 发送 **syscall** 指令
 - 从结果寄存器中取回返回值（如果有）

Service	System Call Code	Arguments	Result
print integer	1	\$a0 = value	(none)
print float	2	\$f12 = float value	(none)
print double	3	\$f12 = double value	(none)
print string	4	\$a0 = address of string	(none)
read integer	5	(none)	\$v0 = value read
read float	6	(none)	\$f0 = value read
read double	7	(none)	\$f0 = value read
read string	8	\$a0 = address where string to be stored \$a1 = number of characters to read + 1	(none)
memory allocation	9	\$a0 = number of bytes of storage desired	\$v0 = address of block
exit (end of program)	10	(none)	(none)
print character	11	\$a0 = integer	(none)
read character	12	(none)	char in \$v0

MIPS程序示例

<u>Label</u>	<u>Op-Code</u>	<u>Dest.</u>	<u>S1</u>	<u>S2</u>	<u>Comments</u>
	Move	\$a0,	\$0		# \$a0 = 0
	li	\$t0,	99		# \$t0 = 99
loop:					
	add	\$a0, \$a0,	\$t0		# \$a0 = \$a0 + \$t0
	addi	\$t0, \$t0,	-1		# \$t0 = \$t0 - 1
	bnez	\$t0, loop			# if (\$t0 != zero) branch to loop #
	li	\$v0,	1		Print the value in \$a0
	syscall				
	li	\$v0,	10		
	syscall				# Terminate Program Run

高级语言的翻译和启动执行

